

รายงาน Assignment EP_03 - ALU (Boolean) [JSIM & VHDL]

1. รายละเอียดการออกแบบ (Detail Design)

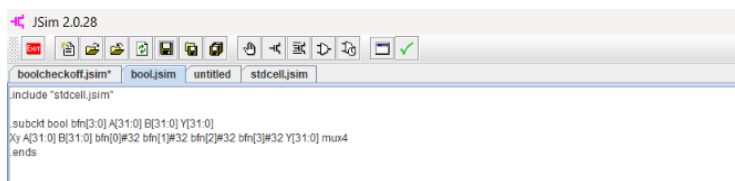
การออกแบบระบบประมวลผล Beta CPU ส่วนของหน่วยคำนวณและตรรกะ (ALU) จะประกอบด้วยโมดูลย่อยหลายส่วน การทดลองนี้เน้นไปที่การออกแบบและจำลองการทำงานของโมดูล BOOL (Bit-wise Boolean) ซึ่งทำหน้าที่ประมวลผลทางตรรกศาสตร์ (Logical Operations) ระหว่างข้อมูลตัวแปร ขนาด 32 บิต จำนวน 2 ชุด คือ อินพุต A และ B หลักการทำงานของโมดูล BOOL อาศัยโครงสร้างของ มัลติเพล็กซ์เซอร์ขนาด 4-to-1 (MUX4) จำนวน 32 ตัวทำงานขนานกันในแต่ละบิต โดยมีการควบคุมดังนี้:

- สัญญาณควบคุม (Function Selection):** รับค่าจากพอร์ต $bfn[3:0]$ ขนาด 4 บิต ซึ่งจะเป็นตัวกำหนดรูปแบบของลอจิกเกต (เช่น AND, OR, XOR) รหัสคำสั่ง 4 บิตนี้จะถูกป้อนเข้าเป็นข้อมูล (Data input) ให้กับขาทั้ง 4 ของ MUX4
- สัญญาณเลือก (Select Lines):** ขาสัญญาณเลือกของ MUX4 จะถูกควบคุมโดยอินพุตบิตที่ i ของ A และ B ($A[i]$, $B[i]$)
- ผลลัพธ์ (Output):** เมื่อสัญญาณ $A[i]$ และ $B[i]$ มีการเปลี่ยนแปลง MUX จะเลือกป้อนค่าบิตที่สอดคล้องกันจาก bfn ออกมาเป็นเอาต์พุต $Y[i]$ ทำให้วงจรนี้สามารถสร้างตารางความจริง (Truth table) ของลอจิกเกตได้ครบทุกรูปแบบ (16 รูปแบบ) ผ่านการเปลี่ยนรหัสคำสั่ง bfn

2. การทำงานของโค้ด JSIM (Gate-Level Simulation)

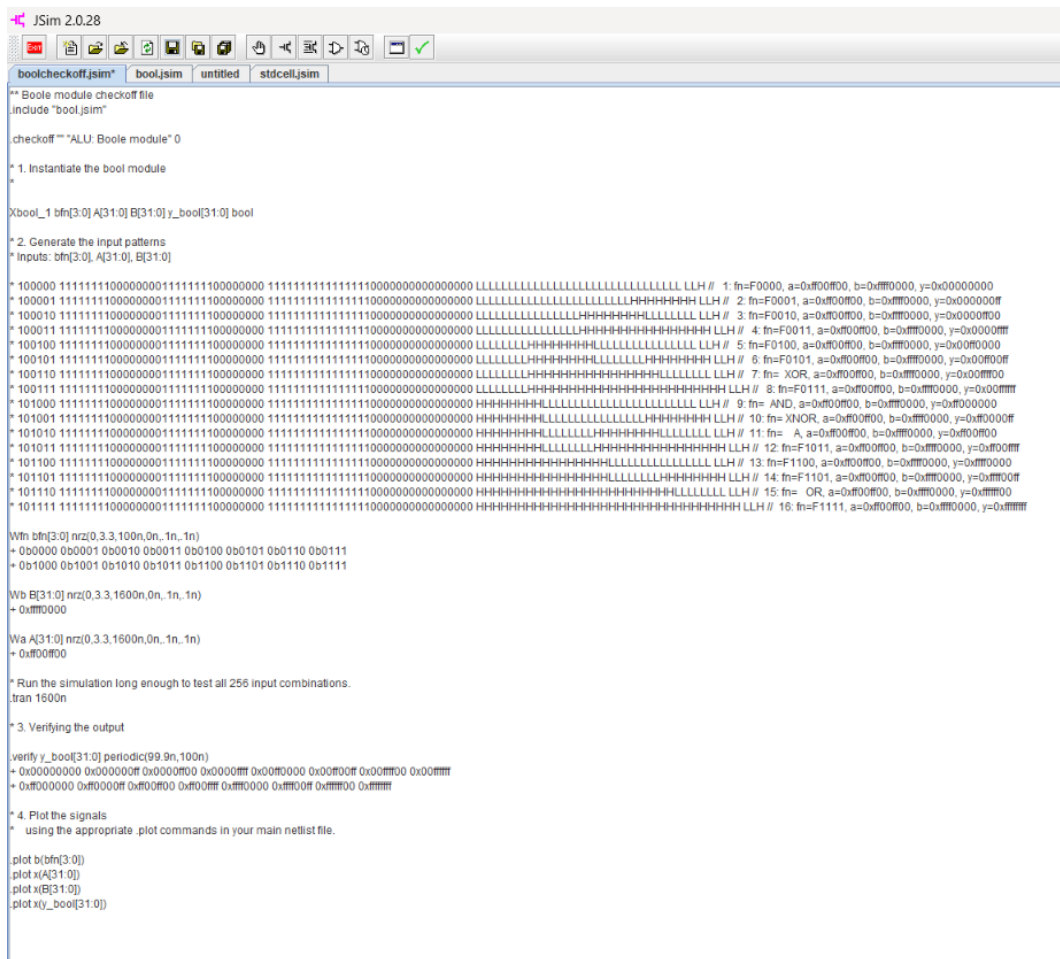
การตรวจสอบการทำงานในระดับโครงสร้างลอจิกเกต (Gate-level) จะทำผ่าน โปรแกรม JSIM โดยมีรายละเอียดของโค้ดดังนี้

2.1 การสร้างโมดูล (ไฟล์ bool.jsim) โมดูลหลักถูกประกาศในรูปแบบของ Subcircuit (.subckt) โดยรับพอร์ตอินพุต bfn[3:0], A[31:0], B[31:0] และให้ผลลัพธ์ที่พอร์ต Y[31:0] ภายในวงจรมีการอ้างอิงไฟล์ .include "stdcell.jsim" เพื่อดึงคอมโพเนนต์พื้นฐานมาใช้งาน จากนั้นทำการเชื่อมต่อ mux4 เข้ากับโครงสร้างบิตทั้ง 32 บิต โดยใช้รหัสคำสั่ง bfn[0] ถึง bfn[3] กระจายแบบขนาน 32 เส้น (#32) เป็นอินพุตข้อมูลให้กับ MUX



2.2 ไฟล์จำลองการทดสอบ (ไฟล์ boolcheckoff.jsim) ไฟล์นี้ทำหน้าที่เป็น Testbench เพื่อสร้างสัญญาณและตรวจสอบความถูกต้อง:

- **การสร้างสัญญาณอินพุต (Waveform Generation):** ใช้ฟังก์ชัน Non-Return-to-Zero (nrz) ในการจำลองรูปคลื่นดิจิทัล โดยกำหนดความกว้างของแต่ละบิตและสร้างช่วงเวลาการเปลี่ยนผ่าน (Transition time) โค้ดได้กำหนดให้ bfn[3:0] ทดสอบครบทั้ง 16 กรณี (ตั้งแต่ 0b0000 จนถึง 0b1111) เพื่อทดสอบฟังก์ชันทางตรรกศาสตร์ทุกรูปแบบ เช่น AND, OR, XOR
- **การตรวจสอบผลลัพธ์อัตโนมัติ:** ใช้คำสั่ง .verify y_bool[31:0] ร่วมกับรูปแบบคาบเวลา periodic(99.9n,100n) เพื่อเปรียบเทียบผลลัพธ์ของวงจรกับค่าคงที่ที่ถูกต้อง (Expected output) ในแต่ละช่วงเวลา (เช่น 0x00000000, 0x000000ff, 0x0000ff00) ตลอดระยะเวลาการซิมูเลชัน .tran 1600n
- **การแสดงผล:** ใช้คำสั่ง .plot ในการดึงรูปคลื่นของ bfn, A, B และ y_bool ออกมาแสดงในหน้าต่าง Waveform Viewer



3. การทำงานของโค้ด VHDL (RTL Simulation)

ในการออกแบบระดับ Register-Transfer Level (RTL) ด้วยภาษา VHDL ได้มีการจัดเตรียมไฟล์ `bool.vhd` ซึ่งกำหนดเอนทิตี (Entity) ของโมดูลให้มีพอร์ต `bfn` (ขนาด 4 บิต), `a` (ขนาด 32 บิต), `b` (ขนาด 32 บิต) และเอาต์พุต `y` (ขนาด 32 บิต) โดยได้ออกแบบ Architecture ไว้ 3 แนวทางเพื่อเปรียบเทียบการสังเคราะห์ (Synthesis) ได้แก่:

3.1 Architecture: beh (Behavioral Model) เป็นการเขียนอธิบายพฤติกรรมของวงจรด้วยคำสั่งสัญญาณพร้อมกัน (Concurrent signal assignment) โค้ดใช้คำสั่ง `for...generate` เพื่อสร้างรูป 32 บิต และใช้เงื่อนไข `when...else` ในการตรวจนับสถานะของ `a(i)` และ `b(i)` เพื่อเลือกบิตของ `bfn(3)` ถึง `bfn(0)` ออกมาเป็นผลลัพธ์ จากนั้นกำหนดให้หน่วงเวลาด้วย `after tpd_mux4`

3.2 Architecture: struct_stdcell (Structural Model) เป็นการเชื่อมต่อวงจรระดับโครงสร้าง โดยทำการเรียกใช้ (Instantiate) เอนทิตี `mux4` จากไลบรารี `stdcell_lib` ที่เตรียมไว้โดยตรง และทำ Port map สัญญาณ `a(i)` และ `b(i)` เข้าที่ขาสัญญาณเลือก `s0, s1` ของมัลติเพล็กซ์เซอร์

3.3 Architecture: fn_stdcell (Function-based Model) เป็นวิธีการเขียนโค้ดโดยเรียกใช้ฟังก์ชัน `mux4()` จากแพ็คเกจ `stdcell_pkg` ซึ่งช่วยลดความซับซ้อนของการเขียน Port map และมีการระบุค่า Propagation Delay ด้วย `after tpd_mux4` ไว้ที่ส่วนท้าย

4. การทำงานของ VHDL Testbench (ไฟล์ bool_tb.vhd)

ในการตรวจสอบโมดูล VHDL จะใช้ไฟล์ bool_tb.vhd ทำหน้าที่เป็น Testbench โดยอาศัยเทคนิคการอ่านข้อมูลจากไฟล์ (File I/O) ซึ่งอธิบายการทำงานได้ดังนี้:

- **การใช้งานไลบรารี:** โค้ดมีการเรียกใช้แพ็คเกจ `std.textio` และ `ieee.std_logic_textio` เพื่อความสามารถในการประมวลผลไฟล์ข้อความ
- **การเชื่อมต่อโมดูลภายใต้การทดสอบ (MUT):** โค้ดทำการเชื่อมต่อ `bool_1`, `bool_2`, และ `bool_3` เข้ากับอินพุตเดียวกัน เพื่อทดสอบสถาปัตยกรรมทั้ง 3 รูปแบบไปพร้อมกัน
- **ฟังก์ชันแปลงค่า (LH_to_01):** มีฟังก์ชันแปลงตัวอักษร 'L' และ 'H' จากไฟล์ Text ให้กลายเป็นสถานะลอจิก '0' และ '1' เพื่อใช้เป็นข้อมูลคาดหวัง (`y_ex`)
- **กระบวนการอ่านและการตรวจสอบ (Process & Assertions):** ลูป `while (not endfile(input_file))` จะทำหน้าที่อ่านไฟล์ `bool_testvec.txt` ที่ละบรรทัดด้วยคำสั่ง `readline` และแยกเก็บข้อมูลลงในตัวแปร `bfm`, `a`, `b` และ `y_exp` จากนั้นโปรแกรมจะหน่วงเวลาจนถึงขอบนาฬิกา (`wait for Tclk-SaT`) แล้วใช้คำสั่ง `assert` เพื่อเปรียบเทียบผลลัพธ์จากวงจรทั้ง 3 ตัว (`y_1`, `y_2`, `y_3`) กับค่าคาดหวัง (`y_ex`) หากลอจิกไม่ตรงกัน ระบบจะแสดงข้อความผิดพลาดระดับ **Severity Warning** ออกมาทันที เมื่ออ่านครบทุกบรรทัด จะรายงานผลว่า "The function verification of BOOL module is successful!"

5. สรุปผลการทดสอบ (Simulation Results)

จากการดำเนินการทดสอบผ่านเครื่องมือ GHDL และดูรูปคลื่นผ่าน GTKWave พบว่าสถาปัตยกรรมการออกแบบทั้ง 3 รูปแบบให้ผลลัพธ์ที่ถูกต้องและตรงกันอย่างสมบูรณ์

รูปคลื่นที่ปรากฏยืนยันให้เห็นว่า เมื่อมีการป้อนคำสั่งลอจิกเข้าไปที่พอร์ต `bfm` เช่น คำสั่ง AND หรือ XOR และกำหนดค่าให้ตัวแปร `a` และ `b` ผลลัพธ์ที่แสดงบนสายสัญญาณ `y_1`, `y_2`, และ `y_3` ให้ผลลัพธ์ทางตรรกศาสตร์ที่ถูกต้องตรงตาม Expected Output ทุกช่วงเวลาการจำลอง โดยไม่มีข้อผิดพลาด

การออกแบบในงานมอบหมายนี้จึงเป็นการยืนยันความถูกต้องของโครงสร้าง MUX4 ในการทำหน้าที่เป็น Bit-wise Boolean Generator สำหรับนำไปรวมกับโมดูลคณิตศาสตร์ในโครงสร้างหลักของ Beta CPU

